DarPOS V 1.0 - User Manual

System Name: DarPOS V 1.0
Version: 1.0
Last Updated: June 22, 2026
Developed by: RICT Consult

Table of Contents

1. System Overview
2. Installation & Setup
3. Core Functionalities
4. User Roles & Permissions
5. Daily Transaction Email Report
6. Reports & Analytics
7. Settings & Configuration
8. Production Deployment
9. Background Jobs & Services
10. Troubleshooting
11. Support

System Overview

What is DarPOS?

DarPOS (Darp Point of Sale System) is a comprehensive web-based Point of Sale (POS) solution built with Laravel 11. It is designed to manage retail operations including sales, purchases, inventory management, expense tracking, and financial reporting.

Key Features

- **Sales Management** - Create, manage, and track sales transactions
- **Purchase Management** - Record and manage supplier purchases
- **Inventory Management** - Real-time stock tracking and adjustments
- **Expense Tracking** - Record and categorize business expenses
- **Daily Email Reports** - Automated daily transaction summaries via email
- **Financial Reports** - Profit, expense, stock, and ledger reports
- **User Management** - Role-based access control with granular permissions
- **Cash Reconciliation** - Daily cash balance verification
- **Dashboard Analytics** - Real-time sales and stock metrics

System Requirements

- **Server:** Linux/Windows with PHP 8.x
- **Database:** MySQL 5.7 or higher
- **Web Server:** Apache/Nginx
- **Browser:** Chrome, Firefox, Safari, Edge (latest versions)
- **PHP Extensions:** BCMath, Ctype, JSON, Mbstring, OpenSSL, PDO, Tokenizer, XML

Installation & Setup

Local Development Setup (XAMPP/WAMP)

Step 1: Clone or Download Project

```
git clone <repository-url>
cd DarpPOS
```

Step 2: Install PHP Dependencies

```
composer install
```

Step 3: Create Environment Configuration

Copy `.env.example` to `.env`:

```
cp .env.example .env
```

Generate application key:

```
php artisan key:generate
```

Step 4: Configure Database

Edit `.env` file:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=darp_pos
DB_USERNAME=root
DB_PASSWORD=
```

Create database:

```
mysql -u root -e "CREATE DATABASE darp_pos;"
```

Step 5: Run Migrations

```
php artisan migrate
```

Step 6: Seed Initial Data

```
php artisan db:seed --class=PermissionTableSeeder
php artisan db:seed --class=CreateAdminUserSeeder
```

Step 7: Start Development Server

```
php artisan serve
```

Access application at: `http://127.0.0.1:8000`

Default Admin Credentials:

- Email: admin@example.com
- Password: password

Core Functionalities

1. Dashboard

Access: Home page after login

What You See:

- Total Purchases (amount)
- Total Sales (amount)
- Stock Balance (quantity)
- Total Stock Value (GHS currency)
- Daily Transaction Report widget (if enabled)
- Monthly Sales Distribution chart

Quick Actions:

- View sales, purchases, products
- Send daily transaction email (manual)
- Access all main features from quick-action buttons

---

2. Sales Management

Creating a Sale

1. Click on **"View Sales"** → **"Create Sale"** button

2. Fill in the following:
    - **Date:** Transaction date (default: today)
    - **Customer:** Select existing or create new customer
    - **Sales Items:** Add products with quantities and prices
    - **Discount:** (Optional) Apply percentage or amount discount
    - **Payment Details:** Record how customer paid (cash, cheque, etc.)
3. Click **"Save & Print"** to finalize

Viewing Sales:

- Click **"View Sales"** to see all sales
- Filter by date, customer, or status
- Edit pending sales (before approval)
- Print invoices
- Cancel sales (if permitted)

---

3. Purchase Management

Creating a Purchase

1. Click on **"Create Purchase"** → **"New Purchase"**
2. Fill in:
    - **Date:** Purchase date
    - **Supplier:** Select supplier
    - **Items:** Add products with costs and quantities
    - **Payment:** Record payment method
3. Click **"Save"**

Managing Purchases

- Click **"View Purchases"** to see all records
- Approve pending purchases
- Track supplier credits
- Print purchase orders

---

4. Inventory Management

Adding/Editing Products

1. Navigate to **Products** → **"Create Product"**
2. Enter:
    - Product name
    - SKU (unique identifier)
    - Category
    - Unit of measurement
    - Cost price
    - Selling price
    - Reorder level
3. Click **"Save"**

Stock Adjustments

1. Go to **Stock Adjustment** → **"New Adjustment"**
2. Select adjustment type (increase/decrease)
3. Add items and quantities
4. Provide reason (damaged, theft, count variance, etc.)
5. Click **"Submit"**

Stock Ledger

View complete stock history:

- Stock Ledger Report: See all movements (purchases, sales, adjustments)
- Product Trail Report: Detailed history per product

---

5. Expense Management

Recording Expenses

1. Click **Expense Voucher → "Create Expense"**
2. Enter:
    - Expense date
    - Expense category (rent, utilities, salaries, etc.)
    - Amount
    - Description/notes
3. Click **"Save"**

Expense Categories

- Manage custom expense categories
- Click **Expense Category → Create/Edit**

Viewing Expense Reports

- **Expense Report:** Summary of all expenses by category
- **Monthly breakdown:** Visualize spending patterns

---

6. User & Role Management

Adding New Users

1. Go to **Users → "Create User"**
2. Fill in:
    - First & Last Name
    - Email address
    - Phone number
    - Assign role (Super Admin, Manager, Cashier, Supervisor)
    - Set password
3. Click **"Create"**

Available Roles

Role	Permissions
Super Admin	Full system access, all features
Manager	Sales, purchases, reports, user management
Cashier	Sales, payments, basic reporting
Supervisor	Monitor staff, reports, approvals

Changing User Permissions

1. Go to **Roles** → Select role
2. Enable/disable permissions for that role
3. Click **"Update"**

---

7. Cash Reconciliation

Daily Cash Reconciliation

1. Click **Cash Reconciliation → "New Reconciliation"**
2. Enter:
    - **Expected Cash:** Total sales minus cheques

- **Actual Cash:** Physical cash counted
- **Cash Shortage/Surplus:** System calculates automatically

3. Add notes if there's a variance
4. Click **"Submit"**

Status Flow:

- **Submitted** → Waiting for manager approval
- **Approved** → Reconciliation accepted
- **Pending Previous Day** → Can't submit until previous day is reconciled

---

Daily Transaction Email Report

What It Does

Sends an automated daily email containing:

- **Total Sales** for the day
- **Total Expenses** for the day
- **Cash Shortage/Surplus** amount
- **Business Date** reference

Email goes to: Company email address set in system

Enabling Daily Email Reports

Step 1: Configure Email Settings

1. Go to **Settings** → **Email Configuration**
2. Toggle **"Enable Daily Email Report"** to ON
3. Select **"Time to Send"** (6 PM to 12 AM, default 7 PM)
4. Click **"Save Settings"**

Step 2: Configure SMTP (Email Server)

Edit `.env` file to use Gmail or your preferred email service:

Gmail Configuration:

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=your-email@gmail.com
MAIL_PASSWORD=your-app-password
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="your-email@gmail.com"
```

To get Gmail App Password:

1. Go to myaccount.google.com
2. Enable 2-Step Verification
3. Generate App Password for "Mail"
4. Use that password in MAIL_PASSWORD

Clear cache after changes:

```
php artisan optimize:clear
```

Step 3: Test Manual Send

1. Go to **Dashboard**
2. Locate **Daily Transaction Report** card
3. Click **"Send Email Now"** button
4. Check email inbox for confirmation

Automatic Daily Sends (Production Only)

On production server, emails send automatically at configured time (default 7 PM) via cron scheduler.

See Production Deployment section for setup.

Email Report Contents

Each email includes:

```
Date: June 22, 2026
Company: Rasdarps Enterprise

├─ Total Sales: GHS 7,610.00
├─ Total Expenses: GHS 0.00
└─ Cash Shortage: GHS 0.00

Status: [Manual/Auto] Send
Recipient: company@example.com
```

Reports & Analytics

Available Reports

1. Profit Report

Shows daily/monthly profit calculations:

- **Gross Profit** = Sales - Cost of Goods Sold
- **Net Profit** = Gross Profit - Operating Expenses

Access: Reports → Profit Report

2. Expense Report

Breakdown of all expenses by category

Access: Reports → Expense Report

3. Stock Reports

- **Stock Report:** Current stock levels and values
- **Stock Ledger:** Complete history of all stock movements
- **Product Trail Report:** Detailed history for each product
- **Stock Value Report:** Total value of inventory

Access: Reports → [Select Report Type]

4. Summary Ledger

Financial overview (Sales + Expenses) by date range

Access: Reports → Summary Ledger

5. Product Price History

Historical pricing changes for products

Access: Products → Product Price History

Settings & Configuration

System Settings

1. Go to **Settings** menu
2. Configure the following:

Setting	Description	Default
Enforce Previous Day Reconciliation	Require cash reconciliation before new sales	OFF

Allow Negative Stock	Permit sales even if stock goes negative	OFF
Prevent Backdated Invoices	Only allow today's date or later	OFF
Allow Manual Discount	Let staff enter custom discounts	ON
Require Password Change	Force password reset on first login	OFF
Enable Daily Email Report	Turn on automated daily emails	OFF
Email Report Time	Hour to send email (18-23, i.e., 6pm-12am)	18 (6 PM)

Company Information

1. Go to **Company**
2. Update:
    - Company name
    - Address
    - Phone
    - Email (for daily reports)
    - TIN number
3. Click **"Update"**

User Profile

1. Click your name → **"Profile"**
2. Update:
    - Name
    - Email
    - Phone
    - Change password
3. Click **"Save"**

Production Deployment

Pre-Deployment Checklist

- [ ] Update `.env` to production values
- [ ] Set `APP_ENV=production`
- [ ] Set `APP_DEBUG=false`
- [ ] Generate production APP_KEY
- [ ] Configure production database
- [ ] Set up email service (SendGrid/Mailgun recommended)
- [ ] Configure backup service
- [ ] Set file permissions (755 for folders, 644 for files)

Step-by-Step Deployment

1. Prepare Server

```
# SSH into production server
ssh user@production-server.com

# Navigate to web root
cd /var/www/html  # or your hosting path

# Clone repository
git clone <repository-url> darppos
cd darppos
```

2. Install Dependencies

```
composer install --optimize-autoloader --no-dev
npm install
npm run build  # for asset compilation
```

3. Configure Environment

```
cp .env.example .env
php artisan key:generate
```

Edit `.env` with production values:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://your-domain.com

DB_HOST=your-db-host
DB_DATABASE=darp_pos_prod
DB_USERNAME=dbuser
DB_PASSWORD=strong-password

MAIL_MAILER=smtp
MAIL_HOST=smtp.sendgrid.net  # or your service
MAIL_PORT=587
MAIL_USERNAME=apikey
MAIL_PASSWORD=your-sendgrid-key
```

4. Database Setup

```
php artisan migrate --force
php artisan db:seed --class=PermissionTableSeeder
php artisan db:seed --class=CreateAdminUserSeeder
```

5. File Permissions

```
chmod -R 755 storage bootstrap/cache
chmod -R 644 storage bootstrap/cache/*
chown -R www-data:www-data storage bootstrap/cache
```

6. Cache & Optimization

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan optimize
```

Background Jobs & Services

Cron Job Setup (For Automated Daily Emails)

The daily transaction email feature requires a scheduler running on your server.

On Linux/Unix Servers

1. **SSH into your server:**

```
ssh user@production-server.com
```

2. **Open crontab editor:**

```
crontab -e
```

3. **Add this line:**

```
* * * * * php /var/www/html/darppos/artisan schedule:run >> /dev/null 2>&1
```

Important: Replace `/var/www/html/darppos` with your actual project path.

4. **Verify cron is running:**

```
crontab -l
```

On Shared Hosting (cPanel/Plesk)

1. Log into hosting control panel
2. Navigate to **Cron Jobs** section

3. Add new cron job with command above
4. Set frequency to "Every Minute"
5. Save

On Windows Servers

Use Windows Task Scheduler:

1. Open **Task Scheduler**
2. Create Basic Task
3. **Name:** DarpPOS Scheduler
4. **Trigger:** Repeat every 1 minute
5. **Action:** Run program
6. **Program:** `C:\PHP\php.exe`
7. **Arguments:** `C:\path\to\darppos\artisan schedule:run`

What the Scheduler Does

Once running, every minute it checks if any scheduled tasks should run:

- **Daily Email:** Runs at configured time (default 7 PM)
- **Other Future Tasks:** Can be added as needed

Verifying Scheduler is Working

1. Check logs:

```
tail -f storage/logs/laravel.log | grep -i schedule
```

2. Look for entries like:

```
[2026-06-22 19:00:00] local.INFO: Daily transaction email sent successfully
```

Troubleshooting

Email Not Sending

Problem: "Error sending email, please try again"

Solutions:

1. **Check Gmail App Password:**
    - Verify `.env` MAIL_PASSWORD is correct
    - Ensure 2-Step Verification is enabled on Google account
2. **Clear Config Cache:**

```
php artisan optimize:clear
```

3. **Test SMTP Connection:**

```
php artisan tinker
Mail::raw('Test', function($m) { $m->to('test@example.com')->subject('Test'); });
```

4. **Check Logs:**

```
tail -f storage/logs/laravel.log
```

Sales Not Showing

Problem: Sales created but not visible

Solutions:

1. Check user permissions: Settings → Roles → Check "sale-list" permission
2. Verify sale status isn't "Draft"
3. Clear cache: `php artisan optimize:clear`

Scheduler Not Running

Problem: Daily emails not sending automatically

Solutions:

1. **Verify cron job exists:**

```
crontab -l  # On Linux
```

2. **Check if cron is running:**

```
sudo service cron status  # On Linux
```

3. **Verify artisan path is correct in cron job**

4. **Check logs for schedule runs:**

```
tail -f storage/logs/laravel.log | grep -i schedule
```

Permission Denied Errors

Problem: "Permission Denied" when accessing features

Solutions:

1. Check user role: Settings → Users → Verify user's role
2. Verify role has permission: Settings → Roles → [Select role] → Enable permission
3. Clear session: Logout and login again

Database Connection Error

Problem: "Connection refused" or database error

Solutions:

1. Verify database is running (MySQL service)
2. Check `.env` database credentials
3. Verify database exists: `mysql -u root -e "SHOW DATABASES;"`
4. Test connection: `php artisan migrate --env=production --force`

Support

Contact Information

RICT Consult

- Phone: +233 242 222 874
- Email: rictconsult@gmail.com
- Website: www.rictconsult.com

Reporting Issues

When reporting issues, include:

1. Error message (exact text)
2. Steps to reproduce
3. User role/permissions
4. Recent `.env` changes
5. Server logs (last 20 lines from `storage/logs/laravel.log`)

Appendix A: Role Permissions Matrix

Feature	Super Admin	Manager	Supervisor	Cashier
Dashboard View	☑	☑	☑	☑
Sales (C/R/U)	☑	☑	✓	☑
Purchases (C/R/U)	☑	☑	✓	✗
Products (C/R/U)	☑	☑	✗	✗
Reports	☑	☑	☑	✓
Cash Reconciliation	☑	☑	☑	✗
Users Management	☑	✓	✗	✗

Roles & Permissions	☑	✗	✗	✗
Settings	☑	✗	✗	✗

☑ = Full Access | ✓ = Limited Access | ✗ = No Access

Appendix B: Database Backup

Manual Backup

```
mysqldump -u root darp_pos > backup_$(date +%Y%m%d_%H%M%S).sql
```

Restore from Backup

```
mysql -u root darp_pos < backup_20260622_160000.sql
```

Scheduled Backups (Production)

Add to crontab:

```
0 2 * * * mysqldump -u root -p'password' darp_pos > /backups/darp_pos_$(date +\%Y\%m\%d).sql
```

Appendix C: Security Best Practices

1. **Always use HTTPS** on production
2. **Change default credentials** immediately
3. **Enable 2-Step Verification** on Gmail/email account
4. **Use strong passwords** (min 12 characters, mixed case, numbers, symbols)
5. **Regular backups** (daily on production)
6. **Monitor logs** for suspicious activity
7. **Keep Laravel updated** with security patches
8. **Restrict file permissions** appropriately
9. **Use environment variables** for sensitive data
10. **Audit user activity** regularly

End of User Manual

Version 1.0 | June 22, 2026 | DarPOS by RICT Consult